Newton-Based Reinforcement Learning

- Reinforcement Learning (RL) enables agents to learn optimal actions.
- Agent interacts with environment to maximize cumulative reward.

Q-Learning

- $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$
- Uses Temporal Difference (TD) error for updates.

Limitations of Q-Learning

- Fixed learning rate
- Slow convergence in sparse reward environments
- Unstable updates

Newton-Based Approach

- Utilizes second-order information by approximating the Hessian, enabling more informed and direction-aware updates compared to standard first-order methods.
- Dynamically adjusts the step size based on the magnitude of the Temporal Difference (TD) error, ensuring stable and efficient learning.
- Achieves quicker and more stable convergence by reducing oscillations and improving update efficiency, especially in sparse reward environments.

Proposed Update Rule

- **Temporal Difference (TD) Error:** $\delta = r + \gamma a' \max Q(s',a') - Q(s,a)$

Measures the difference between the predicted value and the updated target value.

- **Hessian Approximation:** $H \approx |\delta| + \epsilon$

A diagonal approximation of the second-order derivative, used to scale updates adaptively.

- **Newton-Based Q Update:** $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \delta / H$

Adjusts the Q-value using an adaptive step size derived from the TD error.

Advantages

- Adaptive step size
- Improved stability
- Better performance in sparse rewards

Algorithm Steps

- 1. Initialize Q-table
- 2. Choose action
- 3. Compute TD error
- 4. Apply Newton update
- 5. Repeat

Code & Implementation:

```
pip install gymnasium torch torchvision matplotlib numpy jupyter
```

[15] ✓ 0.9s

```
Requirement already satisfied: gymnasium in /Users/ravindramina/.venv/lib/python3.14/site-packages (1.2.3)
Requirement already satisfied: torch in /Users/ravindramina/.venv/lib/python3.14/site-packages (2.10.0)
Requirement already satisfied: torchvision in /Users/ravindramina/.venv/lib/python3.14/site-packages (0.25.0)
Requirement already satisfied: matplotlib in /Users/ravindramina/.venv/lib/python3.14/site-packages (3.10.8)
Requirement already satisfied: numpy in /Users/ravindramina/.venv/lib/python3.14/site-packages (2.4.3)
Requirement already satisfied: jupyter in /Users/ravindramina/.venv/lib/python3.14/site-packages (1.1.1)
Requirement already satisfied: cloudpickle>=1.2.0 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from gymnasium) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from gymnasium) (0.0.4)
Requirement already satisfied: filelock in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (3.25.2)
Requirement already satisfied: setuptools in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (80.9.0)
Requirement already satisfied: sympy>=1.13.3 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torch) (2026.2.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from torchvision) (12.1.1)
Requirement already satisfied: contourpy>=1.0.1 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pyparsing>=3 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: notebook in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (7.4.7)
Requirement already satisfied: jupyter-console in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (6.6.3)
Requirement already satisfied: nbconvert in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (7.16.6)
Requirement already satisfied: ipykernel in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (7.1.0)
Requirement already satisfied: ipywidgets in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (8.1.8)
Requirement already satisfied: jupyterlab in /Users/ravindramina/.venv/lib/python3.14/site-packages (from jupyter) (4.4.10)
Requirement already satisfied: six>=1.5 in /Users/ravindramina/.venv/lib/python3.14/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
pip install --upgrade pip
```

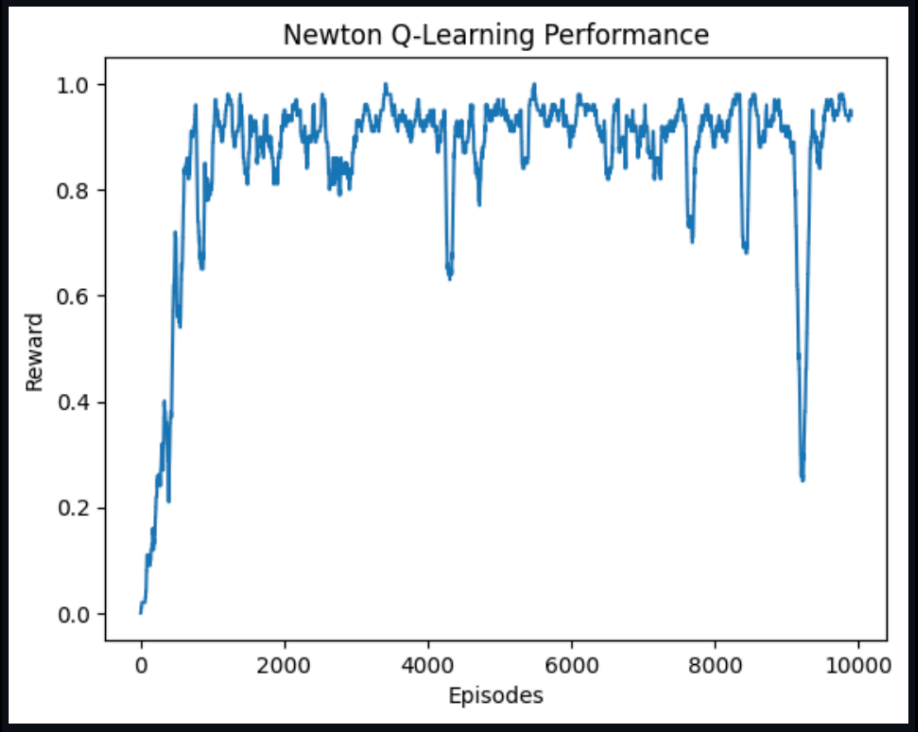
[16] ✓ 1.9s

```
Requirement already satisfied: pip in /Users/ravindramina/.venv/lib/python3.14/site-packages (25.3)
Collecting pip
  Using cached pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-26.0.1-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.3
    Uninstalling pip-25.3:
      Successfully uninstalled pip-25.3
Successfully installed pip-26.0.1
Note: you may need to restart the kernel to use updated packages.
```

```python
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
#environment
env = gym.make("FrozenLake-v1", is_slippery=False)
n_states = env.observation_space.n
n_actions = env.action_space.n
#Q-table
Q = np.zeros((n_states, n_actions))
#hyperparameters
episodes = 10000
alpha = 0.3
gamma = 0.99
epsilon = 1.0   #start high
# Newton update
def newton_q_update(Q, s, a, r, s_next):
    td_error = r + gamma * np.max(Q[s_next]) - Q[s,a]
    grad = td_error
    #better Hessian approximation
    hessian = abs(td_error) + 1e-5
    Q[s,a] = Q[s,a] + alpha * grad / hessian
    return Q

#training
rewards = []

for ep in range(episodes):
    state, _ = env.reset()
    done = False
    total_reward = 0

    while not done:
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[state])
        next_state, reward, done, _, _ = env.step(action)
        Q = newton_q_update(Q, state, action, reward, next_state)
        total_reward += reward
        state = next_state

    #decay epsilon AFTER episode
    epsilon = max(0.05, epsilon * 0.995)
    rewards.append(total_reward)
    if ep % 1000 == 0:
        print(f"Episode {ep}, Reward: {total_reward}, Epsilon: {epsilon:.3f}")

#final Q-table
print("\nFinal Q-table:")
print(Q)

#evaluation
success = 0
for _ in range(100):
    state, _ = env.reset()
    done = False
    while not done:
        action = np.argmax(Q[state])
        state, reward, done, _, _ = env.step(action)
    success += reward
print("\nSuccess Rate:", success / 100)

#plot
window = 100
smoothed = np.convolve(rewards, np.ones(window)/window, mode='valid')
plt.plot(smoothed)
plt.title("Newton Q-Learning Performance")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.show()
```

[25] ✓ 1.1s

```
Episode 0, Reward: 0, Epsilon: 0.995
Episode 1000, Reward: 0, Epsilon: 0.050
Episode 2000, Reward: 1, Epsilon: 0.050
Episode 3000, Reward: 1, Epsilon: 0.050
Episode 4000, Reward: 1, Epsilon: 0.050
Episode 5000, Reward: 1, Epsilon: 0.050
Episode 6000, Reward: 0, Epsilon: 0.050
Episode 7000, Reward: 1, Epsilon: 0.050
Episode 8000, Reward: 1, Epsilon: 0.050
Episode 9000, Reward: 1, Epsilon: 0.050
```




```
Final Q-table:
[[ 0.54312977  0.31570665  0.98540428  0.26334742]
 [ 0.12390512  0.          1.09337136  0.40663169]
 [ 0.49380433  0.75993631  0.01015548  0.17134484]
 [-0.29600997  0.          0.          0.        ]
 [ 0.42379457  0.45359677  0.          0.31337293]
 [ 0.          0.          0.          0.        ]
 [ 0.          1.21185209  0.          0.15800346]
 [ 0.          0.          0.          0.        ]
 [ 0.39643219  0.          0.94748002  0.45508066]
 [ 0.656436    0.67865437  0.84440666  0.        ]
 [ 0.75631207  1.12582165  0.          0.9123374 ]
 [ 0.          0.          0.          0.        ]
 [ 0.          0.          0.          0.        ]
 [ 0.          0.63051069  1.05713135  0.68173813]
 [ 0.45467579  0.64983698  1.16516764  0.59613529]
 [ 0.          0.          0.          0.        ]]

Success Rate: 1.0
```

Results Summary

- High success rate (1.0)
- Stable convergence
- Efficient learning in FrozenLake environment

Comparison

- Compared to standard Q-learning, the proposed Newton-based method achieves faster and more stable convergence.

Conclusion

- Newton-based Q-learning improves convergence and stability.
- Effective for sparse reward problems.

- Name: Ravindra Mina
- Roll No: 25AI60R02
- Course: AI for Cyber Physical Systems